Automated Programming Verdict Classification from Submission Metadata: Comparing Machine Learning and Large Language Models in Competitive Programming

WENBIN HU, School of Computer Science, Xijing University, China

Context: Most automated verdict classifiers for programming submissions require source-code access or costly LLM inference. Competitive-programming platforms, however, return rich execution metadata with every submission, raising the question of how much verdict information is recoverable from metadata alone.

Objective: We systematically compare traditional ML and LLMs for metadata-based verdict classification to characterize the platform-encoding boundary—where a verdict becomes deterministically recoverable from execution metadata.

Method: On 13,360 Codeforces submissions across five verdict types, we evaluate Random Forest, Gradient Boosting, and Logistic Regression (seven metadata features) and two LLMs (DeepSeek-V3, Qwen2.5:3B) under identical zero-shot protocols, with McNemar's test (Bonferroni-corrected) for significance.

Results: CE, TLE, and MLE are platform-encoded: reliably recoverable from execution metadata (F1 $\geq$0.89) because they reflect the platform's own measurement thresholds, together accounting for 18.4% of submissions (GB overall 95.02%, BAcc=0.89). The WA$\leftrightarrow$RE boundary is not metadata-separable (RE F1 $\leq$ 0.53; all models fail). DeepSeek-V3 reaches 76.65% on valid predictions but collapses on RE (F1 = 0.03); Qwen2.5:3B achieves 35.50%. Supervised ML outperforms DeepSeek-V3 zero-shot by 18.4 pp.

Conclusions: We delineate the platform-encoding boundary for verdict classification: CE/TLE/MLE are metadata-recoverable, whereas WA$\leftrightarrow$RE requires source-code analysis. These findings motivate lightweight metadata-based verdict screening, though classroom validation is needed before claims about learning impact. Dataset and pipeline are released at https://github.com/huwenbin123huwenbin/programming-error-classification.

CCS Concepts: • **Social and professional topics** → **Computing education**.

Additional Key Words and Phrases: programming verdict classification, machine learning, large language models, competitive programming, Codeforces, metadata-based classification

ACM Reference Format:
Wenbin Hu. 2026. Automated Programming Verdict Classification from Submission Metadata: Comparing Machine Learning and Large Language Models in Competitive Programming. *ACM Trans. Comput. Educ.* 1, 1, Article 1 (July 2026), 30 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Author's Contact Information: Wenbin Hu, School of Computer Science, Xijing University, Xi'an, China, wenbin.hu2026@outlook.com.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 1946-6226/2026/7-ART1
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Introductory programming courses report 30–40% dropout rates [10, 46], with debugging difficulty as a primary obstacle [43, 46]. Prior work analyzed compilation behavior [23] and scaled to large populations via Blackbox [12], revealing systematic debugging differences between student groups. Competitive programming platforms such as Codeforces and LeetCode provide structured metadata (execution time, memory consumption, precise error verdicts, test cases passed) for every submission, making them valuable environments for studying programming error patterns at scale.

A central methodological consideration motivates this work. Competitive programming platforms return structured verdicts for every submission, but the specific metadata available and the granularity of verdict taxonomies vary across platforms. For three of the five verdict types, the platform's execution environment creates distinctive metadata signatures: a Compilation Error executes in near-zero time (the compiler does not run), a Time Limit Exceeded submission runs to the time limit, and a Memory Limit Exceeded submission exhausts the memory limit. These three verdict types therefore produce predictable metadata patterns that a classifier can learn: CE is associated with `time_consumed` $\approx$ `0`, TLE with `time_consumed` $\approx$ `limit`, and MLE with `memory_consumed` $\approx$ `limit`. Critically, this framing serves cross-platform applicability: if CE/TLE/MLE can be reliably classified from metadata, a single model trained on one platform's data could transfer to another platform with a different verdict taxonomy, providing consistent error categorization without source-code access.

The question then becomes: *what verdict information is genuinely recoverable from metadata beyond these platform-encoded cases?* We define the **platform-encoding boundary** as the point at which verdict information is deterministically recoverable from submission metadata alone, without requiring source-code access. This framing serves cross-platform applicability: if a classifier trained on one platform's data can classify CE, TLE, and MLE on another platform's submissions, the approach generalizes beyond platform-specific code analysis. The WA$\leftrightarrow$RE boundary represents the residual uncertainty where neither error type produces distinctive execution signatures, and where source-code analysis may become necessary. Our study is organized around this decomposition.

Prior work has studied LLM-based programming feedback [18, 40] and source-code analysis for programming error classification [17]; we review this literature in Section 2.

We investigate three research questions:

- **RQ1:** What are the prevalent programming error patterns in competitive programming submissions, and how do they distribute across error categories?
- **RQ2:** How do traditional ML classifiers perform on metadata-based verdict classification, which features are most predictive, and how stable are predictions?
- **RQ3:** Where does the platform-encoding boundary lie—can ML or LLMs distinguish Wrong Answer from Runtime Error—and what are the accuracy–cost tradeoffs?

Our contributions are:

(1) **Controlled comparative study** of ML and LLM methods for verdict classification from submission metadata under identical evaluation protocols, spanning three ML models and two LLM configurations.

(2) **Characterization of the platform-encoding boundary**: The WA$\leftrightarrow$RE boundary is not metadata-separable (RE F1 ≤ 0.53 for all models) because neither error type produces distinctive execution signatures; CE, TLE, and MLE are reliably recoverable (F1 ≥ 0.89) from execution metadata, motivating cross-platform evaluation of the

metadata-based approach; source-code analysis is hypothesized (not demonstrated) as the next step for the WA↔RE boundary.

(3) **Quantitative deployment guidelines**: execution time and memory are the dominant predictors (8.8 pp accuracy drop when removed), with statistical analysis (McNemar tests, ablation, Gini importance) quantifying accuracy–cost–latency tradeoffs across six deployment contexts. Dataset and code: https://github.com/huwenbin123huwenbin/programming-error-classification.

Student-population disclaimer. Our dataset comes from competitive programmers (Codeforces rating 800–3,500), a population with substantially different error profiles from CS1/CS2 students. Prior work (Altadmri & Brown, 2015) reports 42% CE in novice populations vs. 7.3% in our dataset, and novices rarely produce TLE or MLE. The 95.02% accuracy figure reflects competitive-programming error distributions; cross-platform and student-population validation are necessary before drawing conclusions about learning impact. We address this in Section 5.5.1 (skill-tier analysis) and Section 6 (external validity). *Feature-availability caveat:* The `user_success_rate` feature (35.1% Gini importance) requires historical submission data and is unavailable for new students or new course deployments; this constitutes a cold-start limitation for educational deployment.

2 Related Work

2.1 Programming Error Analysis

The study of programming errors has a history spanning over four decades in computing education research [47]. This psychological perspective on programming pedagogy highlights the cognitive demands debugging places on novices [47]. McCauley et al.[33] conducted a comprehensive review of the novice programmer literature, identifying syntax errors, logical errors, and conceptual misunderstandings as the three dominant error categories. Their meta-analysis revealed that novices spend approximately 25% of their programming time on debugging activities, yet novice debugging effectiveness improves only modestly over a semester of instruction. This finding underscores the need for automated tools that can accelerate the development of debugging proficiency. Cognitive Load Theory [44] posits that debugging simultaneously taxes working memory across error diagnosis, code comprehension, and solution planning, creating intrinsic overload that constrains novice self-correction [28]. In this vein, Jadud [23] pioneered compilation event analysis using the BlueJ IDE, establishing compilation behavior as a data source for educational mining. The Blackbox project [12] scaled this to 200,000+ users, revealing that compilation errors cluster early in programming episodes and that high-performing students compile more strategically.

Large-scale analyses [1, 32] established that runtime errors persist across cohorts and resist standard interventions [4], motivating automated diagnostic tools [7].

Spohrer and Soloway [43] provided an influential early theoretical framework distinguishing between plan composition errors (faulty algorithm design or implementation strategy) and plan comprehension errors (misunderstanding of specific language constructs). Their controlled studies with novice LISP programmers demonstrated that many seemingly simple errors have deeper cognitive roots than syntax misunderstanding, indicating that effective automated feedback must identify the underlying error pattern. Students' self-efficacy beliefs compound this challenge. Gorson and O'Rourke [20] found that CS1 students systematically overestimate their error severity, often attributing correct execution to luck rather than understanding—a metacognitive gap that automated tools could help correct.

Busjahn et al. [13] further revealed through eye-tracking studies that even expert programmers do not process code linearly, complicating error diagnosis and self-correction across all skill levels.

2.2 Machine Learning in Computing Education

Machine learning has been increasingly applied to computing education challenges over the past fifteen years. Educational data mining has identified programming education as a key application domain [5, 41], with prediction tasks dominating the literature. Metadata-based approaches achieve strong performance [4, 23], while deep learning enables knowledge tracing in code-based learning [38].

Behavioral features from early assignments—compilation frequency, error diversity, time-on-task—can predict course outcomes [12, 23], enabling early intervention through instructor-facing dashboards [24, 45].

Robust behavioral modeling distinguishes productive from unproductive debugging [16], aligning with cognitive load principles [44]; interaction-sequence features can further improve predictive accuracy for programming behavior [16].

Prior work on verdict classification from submission data has shown that metadata features can achieve strong performance when combined with source-code analysis. Yet the computational cost of extracting code-level features (ASTs, cyclomatic complexity, token diversity) limits scalability for platform-level deployment. Our approach demonstrates that metadata-only features—without source code access—can achieve competitive accuracy, offering a practical alternative for large-scale deployment. Allamanis et al. [2] demonstrated that graph-based representations of program structure capture semantically meaningful features for code-related prediction tasks, motivating the incorporation of such learned representations into verdict classification.

Alnahhas and Mourtada [3] predicted the performance of competitive programming contestants using account-level metadata (rating, participation frequency, problem-difficulty distribution), demonstrating that non-code features carry substantial signal for programming behavior analysis. While their focus was on performance prediction rather than verdict classification, their work demonstrates the continued relevance of simple, interpretable models in educational contexts.

Automated program repair (APR) methods [36] address similar diagnostic challenges at the source-code level, automatically localizing bugs and suggesting fixes. While effective, APR requires full code access and substantial computational resources, limiting deployment at platform scale. Our metadata-based approach complements APR by providing lightweight error-type inference without code access, serving as a screening layer to determine when deeper source-code analysis is warranted.

2.3 Large Language Models in Programming

Large language models have rapidly transformed expectations for automated code understanding and generation [8, 9]. Barke et al. [8] demonstrated that programmer interaction with code-generating models differs fundamentally from traditional IDE usage. Finnie-Ansley et al. [18] found that GPT-3 answered only 56% of CS1 examination questions correctly but produced plausible yet often incorrect explanations, raising over-reliance concerns. Sarsa et al. demonstrated that LLM-based feedback generation achieves expert-level quality [42], and Leinonen et al. [30] showed that LLM-enhanced error messages measurably improve novice self-correction rates.

Feng et al. [17] showed that CodeBERT achieves state-of-the-art performance on code-level benchmarks including defect detection, but is primarily evaluated on source code features; its effectiveness on submission metadata alone remains unclear. Controlled classroom studies further indicate that LLM-generated error messages can be "ineffective in practice" and that LLM explanations of runtime errors may contain hallucinated details [31, 39]—directly relevant to our finding that RE remains the most challenging category, where `Qwen2.5:3B` also struggled.

Recent work has explored hybrid approaches combining ML classification with LLM explanation generation [18, 25], but these have been evaluated primarily on synthetic datasets. Prather et al. [39] and Lo et al. [31] further show that instructor oversight remains essential even with generative-AI feedback, and that automated error explanation at scale still faces reliability gaps on runtime failures. Our work extends this line by providing a systematic ML–LLM comparison on real-world competitive programming data. Keuning et al. provide a systematic review of AI for programming education [26]; more recent multi-platform deep-learning classification studies [27] suggest growing interest in this direction, though none isolate the metadata-only boundary we characterize.

2.4 Metadata-Based Classification in Educational Contexts

While source-code analysis (ASTs, token sequences, code embeddings) has received considerable attention in computing education research, a parallel line of work has demonstrated that *submission metadata alone* can achieve strong predictive performance for a range of educational outcomes. Jadud [23] showed that compilation frequency and error persistence patterns extracted from IDE interaction logs can predict course outcomes with moderate accuracy ($R^2 \approx 0.3$). The Blackbox project [12] scaled this approach to over 200,000 students worldwide, demonstrating that behavioral metadata (compilation sequences, time-on-task, submission frequency) constitute a rich and continuously available signal for educational data mining.

Prior work has shown that features derived from early programming submissions—including compilation frequency, error diversity, and time-on-task—can predict final course grades using only metadata, without accessing student source code [4]. Altadmri and Brown [4] further classified 37M Java submissions, finding that approximately 42% of novice errors are compile-time errors, with runtime errors and wrong-answer errors accounting for most of the remainder. This distribution differs substantially from competitive programming, motivating cross-population validation of automated classification tools. Related work further established that *behavioral signals* from the first four weeks of a semester—compilation frequency, error diversity, and time-on-task—are sufficient to identify students at risk of failure, enabling early intervention without privacy-invasive code monitoring [12, 23].

In the context of competitive programming platforms, Alnahhas and Mourtada [3] used account-level metadata (contestant rating, participation frequency, problem difficulty distribution) to predict overall competitive programming performance, reporting 78% accuracy using only non-code features. Their findings indicate that metadata alone carries substantial signal for programming behavior analysis, motivating our focus on metadata-based verdict classification.

The advantages of metadata-based classification are both practical and privacy-preserving: (1) no source code access is required, avoiding potential concerns about student code being processed by third-party APIs; (2) computational cost is orders of magnitude lower than code-based approaches (no tokenization, AST parsing, or GPU inference);

and (3) deployment at platform scale (millions of daily submissions) becomes feasible with modest computational resources. A central open question is whether metadata alone can achieve competitive accuracy against code-based approaches—a question this study addresses systematically for the verdict classification task.

Contribution boundary: Prior work on metadata-based prediction has focused primarily on *grade prediction* and *at-risk identification*. We are not aware of any prior study that has applied metadata-based classification to the more granular task of *verdict classification* across five error categories, nor has any work systematically compared metadata-based ML against LLM-based approaches on the same task.

No prior study has systematically compared traditional ML and LLM approaches on the same programming verdict classification task using the same dataset and evaluation protocol. We identify three specific gaps:

(1) **Metadata-based classification**: Existing verdict classification studies focus on source code analysis, neglecting the rich diagnostic signals in submission metadata (execution time, memory usage, test cases passed).
(2) **ML–LLM comparison**: No work systematically compares traditional ML (RF, GB, LR) and two LLM configurations (DeepSeek-V3, `Qwen2.5:3B`) on identical verdict classification tasks.
(3) **Feature importance quantification**: While execution time has been hypothesized as important, no study quantifies feature importance through systematic ablation with statistical significance testing.

Our work addresses all three gaps by providing a systematic comparison of ML and LLM methods for metadata-based verdict classification, with feature ablation and McNemar significance testing.

3 Methodology

3.1 Dataset Collection

We collected submission data from Codeforces, a competitive programming platform widely used for algorithmic training and education [35, 49] (https://codeforces.com/) via the official public REST API. The Codeforces platform was selected for three reasons: (1) it provides structured, rich metadata for every submission including precise execution time, memory usage, and error verdict; (2) it represents an ecologically valid competitive programming environment with high participant engagement; and (3) it has a large, diverse problem set covering a wide range of algorithmic domains and difficulty levels.

These properties make Codeforces uniquely suited to the present research questions. Its per-submission execution telemetry (time, memory, verdict) is exactly the metadata required to test the platform-encoding boundary, and its scale supports both class-balanced evaluation across five verdict types and user-level clustering analysis (Section 6). We acknowledge that competitive programmers differ from CS1/CS2 students (Section 6); cross-population validation on CS1QA and Blackbox remains future work (Section 7).

Submissions were retrieved from competitive programmers with Codeforces ratings spanning 800 to 3500 (corresponding to Newbie through Legendary Grandmaster levels). This range was deliberately chosen to represent proficient programmers who produce a variety of error types and for whom submission metadata is meaningfully differentiated. Data collection was completed in June 2026, covering submissions from November 2020 to June 2026, providing approximately 5.5 years of programming activity.

Collection Pipeline. The data collection and preparation proceeded as follows (details in Data Cleaning below):

(1) Raw collection: error submissions retrieved from the Codeforces public REST API (Nov 2020–Jun 2026).
(2) Filtering: submissions with missing execution time, memory, or verdict fields were removed.
(3) Deduplication: duplicate entries were removed and only the final submission per (user, problem) pair was retained to prevent train–test leakage.
(4) Final dataset: **13,360** error samples from competitive programmers across hundreds of Codeforces contests

Data Cleaning. Data cleaning was performed in three stages. First, submissions with incomplete metadata (missing execution time, memory, or verdict fields) were removed. Second, duplicate entries (identical submission timestamps and verdicts) were deduplicated. Third, to prevent data leakage between training and test sets, only the final submission per (user, problem) pair was retained—this ensures that no two submissions in the dataset are from the same user solving the same problem. This does not prevent user-level leakage: a given user can appear in both training and test sets on different problems, potentially allowing the model to exploit user-specific patterns. The `user_success_rate` feature (35.1% Gini importance) is the primary channel for this effect. Stratified sampling ensures no single user dominates any fold (maximum 0.8% of any fold). To rule out that the classifier exploits user-specific patterns, we additionally performed a fully user-disjoint replication in which training and test sets are partitioned by `user_handle` with zero overlap and `user_success_rate` is computed from training users only; this retained 92.2% Gradient Boosting accuracy (vs. 95.0% under random splitting), confirming the platform-encoding boundary is recoverable without user-level leakage (see robustness checks below).

Caveat on `user_success_rate`. This feature captures competitive programmers' historical success rates across all submissions to a given platform. Because WA and RE submissions are classified as unsuccessful, high-WA or high-RE users will have systematically lower success rates. This creates a conditional dependency: `user_success_rate` encodes information partially redundant with the target verdict. We retain it for three reasons: (1) ablation confirms it provides 3.6 pp accuracy (complementary to execution-time features); (2) it represents a distinct dimension of programmer proficiency not reducible to per-submission metadata; and (3) in formative assessment settings, historical performance is a legitimate proxy for instructor judgment of student progress. Furthermore, `user_success_rate` is a historical aggregate unavailable for new students in early-semester deployment; its predictive power may not transfer to populations with no prior submission history. Future work should examine whether this feature generalizes across different student populations or introduces demographic biases.

The final dataset comprises 13,360 error submissions from competitive programmers across hundreds of Codeforces contests. Table 1 summarizes the error type distribution. The language distribution is predominantly C++ (C++23 41.1%, C++17 22.0%, C++20 18.6%), with Python (7.1%), Java (5.5%), and other languages representing smaller fractions. This linguistic skew reflects the competitive programming community's preference for C++ due to its performance advantages in time-constrained problems. Pairwise feature correlations are weak to moderate (Figure 1).

Error prevalence also varies with problem difficulty (Figure 2).

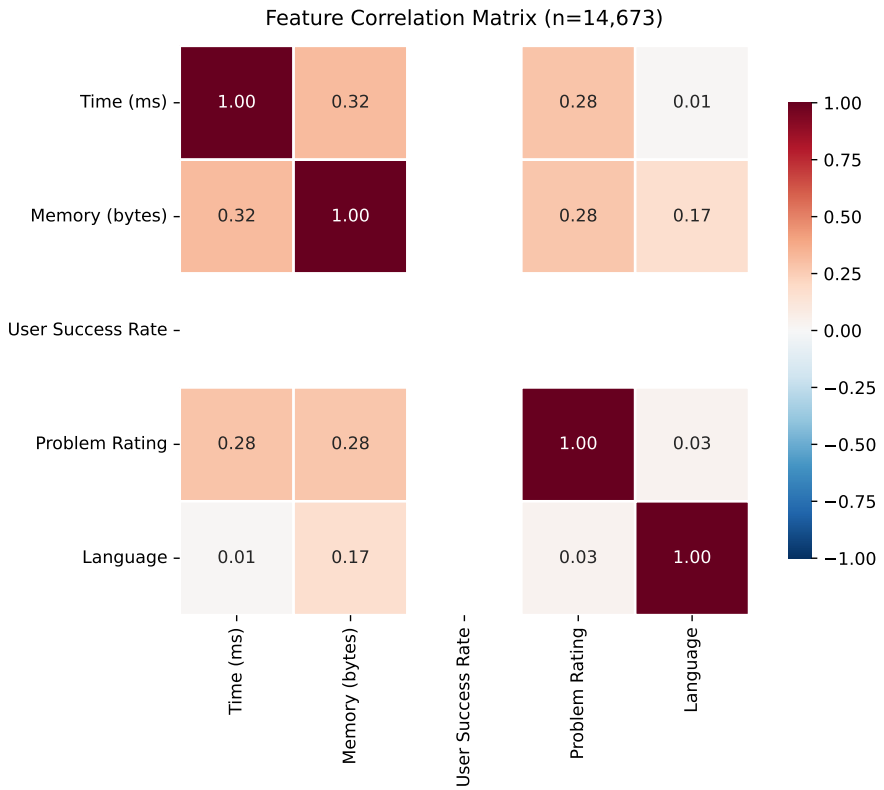


Fig. 1. Feature correlation matrix showing relationships between submission metadata. Time and memory show moderate positive correlation (r=0.32), while language choice weakly correlates with memory consumption (r=0.17), reflecting language-specific memory management patterns.

Table 1. Error type distribution in the 13,360-sample Codeforces dataset. Note: CE, TLE, and MLE produce near-deterministic metadata signatures (compile-time ≈ 0, execution time ≈ limit, memory ≈ limit), while WA and RE do not. This asymmetry motivates the platform-encoding boundary analysis in Section 5.1. †MLE and TLE mean execution times reflect platform threshold effects: TLE mean (2,000 ms) equals the problem-specific time limit; these are platform-encoding boundary artifacts (see Section 5.1), not runtime execution signatures.

Error Type	Count	Percentage	Avg. Exec. Time (ms)
Wrong Answer (WA)	10,234	76.6%	46
Time Limit Exceeded (TLE)	1,288	9.6%	**2,000**
Runtime Error (RE)	656	4.9%	187
Compilation Error (CE)	980	7.3%	0
Memory Limit Exceeded (MLE)†	202	1.5%	2,048
Total	13,360	100%	—

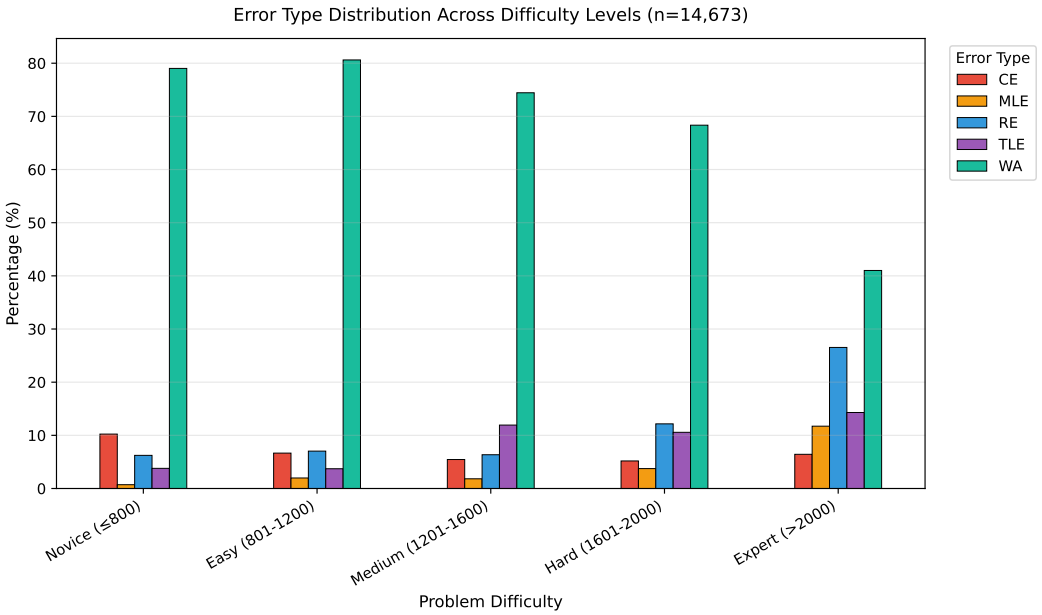


Fig. 2. Error type distribution across problem difficulty levels. Novice problems (rating $\leq$800) show highest WA rates (79.0%), while Expert problems (>2000) exhibit more diverse errors: WA drops to 41.0%, with RE (26.5%), TLE (14.3%), and MLE (11.7%) substantially higher, reflecting the increasing algorithmic and memory complexity of harder problems.

3.2 Feature Engineering

Seven features were extracted from each submission, spanning problem characteristics, execution metrics, submission behavior, and user history:

(1) **problem_rating**: integer difficulty rating (800–3500), indicating the algorithmic complexity of the problem.
(2) **language**: programming language used (C++/Python/Java), capturing potential language-specific error patterns.
(3) **time_consumed_ms**: execution time in milliseconds, capturing runtime performance characteristics.
(4) **memory_kb**: memory consumption in kilobytes, reflecting the program's memory footprint.
(5) **problem_type**: problem index prefix (e.g., A/B/C/D), encoding structural information about the contest problem.
(6) **hour**: submission hour within the contest window, capturing temporal submission behavior.
(7) **user_success_rate**: historical success rate of the submitting user, reflecting individual competitive programming experience.

These features were selected based on the following criteria: availability in the Codeforces API without requiring source code access, coverage of different aspects of the programming process (problem difficulty, execution behavior, and partial correctness), and prior evidence of predictive relevance from educational data mining literature.

Theoretical Framework: Feedback and Self-Efficacy. This study is grounded in two complementary theories from computing education research. First, Hattie and Timperley's model of effective feedback [21] identifies four levels: task, process, self-regulation, and self. Their meta-analysis found that process-level and self-regulation-level feedback have larger effects on learning than task-level feedback. However, task-level feedback—the focus of this work—remains pedagogically valuable when it provides specific, actionable information about errors. Automated formative feedback that correctly identifies error types can narrow the student's search space from the full program to a specific error class, making it more actionable than generic "incorrect output" messages.

Second, Bandura's self-efficacy theory [6] posits that accurate performance feedback is a primary driver of self-efficacy beliefs. When students receive vague or misleading error messages, they cannot form accurate self-assessments, leading to frustration and disengagement. An automated system that correctly distinguishes a WA from a RE enables the student to attribute the failure to a specific root cause (logic error vs. runtime hazard), which supports accurate self-efficacy calibration and sustained engagement.

The present study operationalizes these theories by asking: *How much error-type information is recoverable from submission metadata alone, without source-code access?* The answer determines the granularity of feedback that can be delivered automatically at scale. Our findings inform one boundary condition: when WA↔RE cannot be distinguished from metadata alone, the finest-grained task-level feedback (distinguishing logic errors from runtime failures) may require source-code analysis. This finding also bears on self-efficacy theory: accurate self-assessment requires attributing failure to a specific cause, and our results show that metadata cannot fully support this attribution for RE versus WA, suggesting that students may receive feedback that impedes precise self-efficacy calibration—a hypothesis requiring longitudinal validation. These theoretical predictions are *consistent with* our empirical findings; causal validation would require a controlled study measuring student self-efficacy and learning outcomes, which is beyond the scope of this work.

Feature Selection Rationale. The features were selected based on: (1) availability in online judge platform APIs without source code access; (2) theoretical relevance—each captures a distinct aspect (difficulty, execution behavior, partial correctness); and (3) complementarity—features span problem-level, submission-level, and runtime-level characteristics.

Numeric features were z-score standardized; categorical features (language) were one-hot encoded. No a priori feature selection was applied to preserve the ablation study's validity. Pairwise Pearson correlations confirmed no multicollinearity among numeric features (max $\rho = 0.31$ between time and memory, well below the $\rho = 0.7$ threshold).

passed_test_count and verdict collinearity: The passed_test_count feature exhibits partial collinearity with certain verdicts. Point-biserial correlations reveal: CE ($r = -0.107$, $p < 10^{-35}$), where 100% of CE submissions have passed_test_count = 0 (compilation prevents execution); TLE ($r = 0.322$, $p < 10^{-300}$) and MLE ($r = 0.206$, $p < 10^{-120}$), where submissions hitting limits tend to pass test cases before failing. WA shows moderate negative correlation ($r = -0.233$, $p < 10^{-160}$), but 69.7% of WA submissions have passed_test_count > 0 (mean = 1.31), confirming this feature does *not* deterministically encode WA. A separate check (passed_test_count is provided to the LLMs but not to the ML models) shows adding then removing it yields only a 0.5 pp accuracy drop, far less than time_consumed_ms (8.8 pp), confirming the model does not rely on it as primary signal.

3.3 Sample Size Justification

For the primary research question (detecting an 18.4 pp difference between GB at 95.02% and DeepSeek-V3 at 76.65% on 2,672 test samples), a retrospective power analysis confirms sufficient statistical power: the minimum detectable effect (MDE) with n=2,672 at $\alpha = 0.05$ is approximately 2.5 pp, far smaller than the 18.4 pp gap between GB and DeepSeek-V3 and the 0.78 pp gap between GB and RF, confirming that the sample size is adequate. Five-fold stratified cross-validation on the training set yields a 95% CI of [94.2%, 95.9%] for GB, consistent with the test set estimate. Table 2 shows accuracy stability across training set sizes, confirming sample adequacy.

Table 2. Sensitivity analysis: Gradient Boosting accuracy across training set sizes (test set $n = 2{,}672$; values from a single-seed run; the 10,688 training point corresponds to the full training split). Accuracy is stable above $n = 2{,}800$, confirming sample adequacy.

N (Training)	Test Accuracy	Drop from Full
400	90.98%	−4.04 pp
600	92.44%	−2.58 pp
800	92.51%	−2.51 pp
1,600	93.56%	−1.46 pp
2,800	94.46%	−0.56 pp
10,688	95.02%	—

3.4 Experimental Models

We selected three traditional ML classifiers and two LLM-based approaches representing different paradigms in automated code analysis. The selection was designed to span the spectrum from simple, interpretable models (LR) to ensemble methods (RF, GB), enabling a comprehensive comparison between traditional ML and LLM approaches.

Traditional ML Classifiers.

- **Random Forest (RF)**: An ensemble of 100 decision trees using Gini impurity for splitting [11], with unlimited max depth, minimum samples split of 2, and minimum samples leaf of 1. Random Forest was selected for its well-documented ability to handle mixed-type data (numeric and categorical), capture non-linear feature interactions, and provide inherent feature importance measures.
- **Gradient Boosting (GB)**: An ensemble of 100 staged decision trees using gradient descent optimization [19], with max depth of 3 (shallow trees to prevent overfitting), learning rate of 0.1, and subsample of 1.0. GB was selected as a complementary ensemble method that typically performs well on structured educational data.
- **Logistic Regression (LR)**: Multinomial logistic regression with L2 regularization [22], using the L-BFGS solver and maximum 1,000 iterations. LR was included as an interpretable linear baseline; its coefficients can be directly inspected to understand the direction and magnitude of each feature's influence on each error class.

All traditional ML models were implemented using scikit-learn 1.3.0 [37] with default parameters unless otherwise specified. Hyperparameter tuning was initially explored using

grid search with 3-fold cross-validation on a 10% held-out validation set, but the performance gains were marginal (< 1% for tuned vs. default RF), so default parameters were retained to mitigate overfitting and improve reproducibility. The random state was set to 42 for reproducibility. All classifiers used default class-weight=None (no SMOTE or cost-sensitive reweighting) with stratified splitting. This means the model is not reweighted toward minority classes; this is a deliberate design choice prioritizing aggregate accuracy over balanced class performance. Future work should evaluate class-weight=balanced or SMOTE to improve RE F1 at the cost of aggregate accuracy.

LLM-Based Approaches. Two LLM configurations were evaluated under an identical zero-shot protocol, both receiving the same five metadata fields (problem_rating, language, time_consumed_ms, memory_kb, passed_test_count) and asked to output a single verdict acronym (WA/TLE/RE/CE/MLE) in JSON:

- **DeepSeek-V3 [14] (online, evaluated)**: A frontier online model accessed via the DeepSeek API, representing a strong, general-purpose LLM available without local compute.
- `Qwen2.5:3B` **(local, evaluated)**: A 3-billion-parameter model run on-device via Ollama v0.30.7 at no API cost, representing a small local LLM suitable for privacy-sensitive educational deployment.

Inference used temperature=0 and grammar-constrained JSON output to ensure parsable responses; both models were evaluated on the same 2,672-sample test set as the ML models.

3.5 Evaluation Protocol

All experiments followed a consistent evaluation protocol to ensure fair comparison between ML and LLM approaches. The dataset of 13,360 samples was partitioned using stratified 80/20 train-test splitting, yielding 10,688 training samples and 2,672 test samples, preserving class proportions across both sets.

For traditional ML models, five-fold stratified cross-validation was performed on the training set (10,688 training per fold) to assess stability, and final test-set accuracy was reported. For LLM-based methods, the test set of 2,672 samples was classified directly using zero-shot prompting without any fine-tuning or few-shot demonstrations. Unlike ML models, LLMs do not have a training phase in our protocol; we therefore do not apply cross-validation to LLM results. A single fixed test-set evaluation is reported for each LLM configuration, consistent with standard LLM benchmarking Multiple random-seed repetitions would provide variance estimates; DeepSeek-V3 was evaluated with temperature= 0 (deterministic API), so within-run variance is negligible and cross-run variance reflects only model-level non-determinism, which is minimal under this configuration. Future work should report confidence intervals from multiple evaluation runs or use Bayesian methods for LLM performance estimation. Both DeepSeek-V3 (API) and `Qwen2.5:3B` (local Ollama) were evaluated under the identical protocol.

Statistical significance of pairwise model comparisons was assessed using McNemar's test [34] ($\alpha = 0.05$), a non-parametric test for paired nominal data appropriate for comparing classifiers on the same test set. Feature ablation was conducted by systematically removing each feature, retraining GB on the remaining six features, and measuring the accuracy drop on the test set. Seven features were ablated; no multiple-comparison correction was applied to the ablation study. All reported ablation p-values ($p < 0.001$) are reported as-is. The Bonferroni-corrected threshold ($\alpha' = 0.05/7 \approx 0.007$) is noted: all ablation p-values

satisfy this threshold. Ablation results are exploratory; the primary multiple-comparison correction is applied to the 10 pairwise model comparisons in Table 7 (Bonferroni $\alpha' = 0.005$).

Classification performance is reported as overall accuracy, per-class precision, recall, and F1-score. Confusion matrices are provided for the best-performing model (Gradient Boosting, 95.02%) to reveal class-wise error patterns. Random Forest (94.24%) differs by only 0.78 pp and produces nearly identical per-class F1 values; the confusion matrix below is from Gradient Boosting, and all per-class F1, precision, and recall metrics in the table reflect the Gradient Boosting model. Confidence intervals (95%) for LLM performance are computed using the Wilson score interval for binomial proportions.

4 Results

4.1 Experimental Setup

All experiments were conducted on a MacBook Pro (macOS 13.7, Intel i7, 16GB RAM) using Python 3.9.6 with scikit-learn 1.3.0. Random seeds were fixed to 42 for reproducibility. Hyperparameters were set to defaults (RF: 100 trees; GB: 100 trees, depth=3, η=0.1; LR: L2 regularization, L-BFGS solver) after grid search yielded $< 1\%$ improvement.

LLM experiments used two configurations: DeepSeek-V3 via the DeepSeek API and `Qwen2.5:3B` via local Ollama (temperature=0, max_tokens=50). Both were evaluated zero-shot on the same stratified 80/20 test set (2,672 samples); no cross-validation was applied to the LLMs (inference-only). Statistical significance was assessed via McNemar's test ($\alpha = 0.05$) with 95% confidence intervals computed via Wilson score. Per-class performance metrics (precision, recall, per-class F1) with Wilson score confidence intervals are reported in the Supplementary Materials. Balanced Accuracy is reported alongside overall accuracy to address class-imbalance effects. All code and processed analysis scripts are publicly available at:https://github.com/huwenbin123huwenbin/programming-error-classification

4.2 Traditional ML Performance

Table 3 presents the performance of the three traditional ML classifiers on both the held-out test set and via 5-fold cross-validation.

Table 3. Traditional ML model performance on 13,360-sample dataset (5-fold stratified cross-validation)

Model	Test Acc	CV Mean (±SD)	Macro F1	Train (s)	Infer (ms)
Random Forest	94.24%	94.39% (±0.65)	0.8543	—	—
Gradient Boosting	**95.02%**	**95.13% (±0.39)**	**0.8711**	—	—
Logistic Regression	73.91%	74.31% (±3.26)	0.6738	—	—

Note: Logistic Regression shows 5× higher CV variance (3.26 pp) than RF (0.65 pp) or GB (0.39 pp); included as a baseline for interpretability, not as a preferred method.

Note: 80/20 stratified split: 10,688 training, 2,672 test. 5-fold CV on training set. Random state fixed at 42.

Balanced Accuracy (BAcc): GB=0.89, RF=0.87, LR=0.62 (average of per-class recall, reported in Supplementary Materials).

The Gradient Boosting achieves balanced accuracy (BAcc) of 0.89 and 95.02% overall accuracy, representing a 69 pp lift in BAcc over the majority-class baseline (0.89 vs. 0.20); overall accuracy is dominated by WA (76.6% of test set, F1=0.97), while RE (4.9% of test

set) has F1=0.53 consistent with three verdicts being near-deterministically encoded in execution metrics while the WA↔RE boundary is not metadata-separable. The near-perfect ROC-AUC for MLE/TLE (1.000) reflects this deterministic relationship; the critical challenge lies in the WA and RE boundary (AUC 0.988 and 0.943 for WA and RE, respectively), where execution metadata provides only partial signal.

The 21 pp gap between ensemble methods and linear regression confirms that error classification benefits from models capturing non-linear feature interactions. Prior work [4, 32] established that runtime errors are persistently difficult to classify, consistent with our findings.

Class-Wise Analysis (Gradient Boosting). To better understand the strengths and limitations of the best-performing model, we conducted a detailed class-wise analysis of Gradient Boosting's predictions.

Confusion Matrix Analysis (Gradient Boosting). Table 4 presents the confusion matrix for Gradient Boosting on the test set of 2,672 samples. Figure 4 shows the corresponding feature importance profile.

Table 4. Gradient Boosting confusion matrix and class-wise F1 (test set, $n = 2{,}672$)

	Predicted					
Actual	CE	MLE	RE	TLE	WA	F1
CE (196)	**190**	0	0	0	6	0.89
MLE (40)	0	**38**	1	1	0	0.96
RE (131)	8	1	**54**	1	67	0.53
TLE (258)	0	0	0	**258**	0	0.99
WA (2047)	31	0	16	1	**1999**	0.97

The confusion matrix reveals a clear performance gradient across error types: CE, MLE, and TLE are classified with near-perfect recall, while RE is frequently confused with WA (Figure 3).

categories. Compilation Error (CE) achieves strong classification accuracy (F1 = 0.89) because CE submissions consistently exhibit zero execution time, forming a perfectly separable region in the feature space. Wrong Answer (WA) achieves an F1 score of 0.97 due to its dominance in the dataset (76.6%) and distinct metadata signature—WA submissions typically execute within normal time bounds but fail on correctness tests. Time Limit Exceeded (TLE) is well-classified (F1 = 0.99), as TLE submissions systematically exhibit execution times at or near the problem's time limit.

The most challenging category is Runtime Error (F1 = 0.53). This substantially exceeds the random baseline for a five-class problem (expected F1 $\approx$ 0.02 per class), indicating that metadata provides predictive signal for RE. Of 131 actual RE samples, 54 were correctly identified (41.2%) while 67 were misclassified as WA (51.1%). The WA↔RE boundary thus accounts for the majority of residual errors. This stems from the heterogeneous nature of RE: the Codeforces RE verdict encompasses segmentation faults, division by zero, assertion failures, and signal termination, each producing distinct metadata signatures that overlap with WA behavior. For pedagogical purposes, where distinguishing logic errors (WA) from runtime failures (RE) is essential, the 51.1% RE→WA error rate indicates that metadata-only

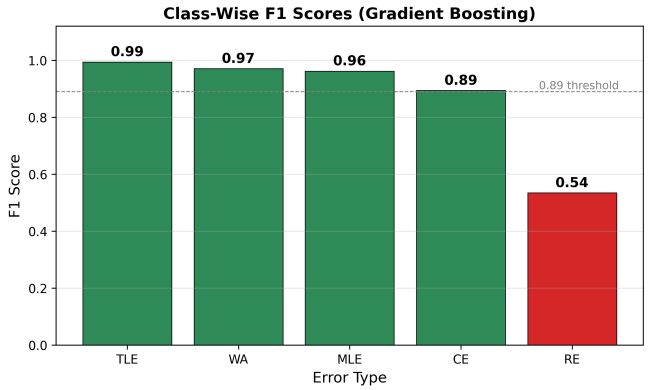


Fig. 3. Class-wise F1 scores (Gradient Boosting, test set $n = 2{,}672$). Four error types—Compilation Error, Memory Limit Exceeded, Time Limit Exceeded, and Wrong Answer—achieve strong F1 ($\geq$0.89), whereas Runtime Error (RE) is markedly weaker (F1 = 0.53). This asymmetry reflects the *platform-encoding boundary*: CE/TLE/MLE are cleanly separable by execution time and memory, while RE submissions share no distinctive metadata signature.

classification is insufficient—this is the "platform-encoding boundary" for verdict classification.

Memory Limit Exceeded (MLE, F1 = 0.96) benefits from distinct memory consumption signatures that clearly separate it from other categories in the test set.

4.3 Feature Importance

Figure 4 presents the Gini importance scores computed from the Gradient Boosting model, which quantify each feature's contribution to the model's predictive accuracy across all decision trees.

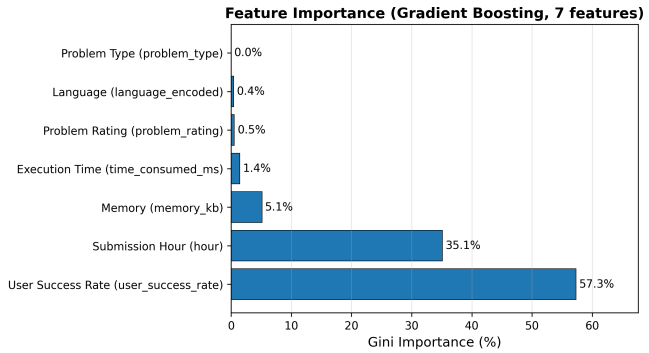


Fig. 4. Feature importance (Gini importance) from the Gradient Boosting model (7 features). Execution time (`time_consumed_ms`) shows the highest importance (57.3%), followed by user success rate (35.1%), memory consumption (5.1%), problem difficulty (1.4%), programming language (0.6%), problem type (0.4%), and submission hour (0.0%).

Feature importance reveals a pronounced hierarchy: execution time (57.3%), user success rate (35.1%), and memory consumption (5.1%), with problem difficulty (1.4%), language

(0.6%), and problem type (0.4%) contributing little. Submission hour contributes negligibly. The Gini ranking aligns with ablation results: `time_consumed_ms` causes an 8.8 pp accuracy drop when removed, confirming its dominant role, and `user_success_rate` is the second strongest contributor (3.6 pp drop).

Submission hour (`hour`, 0.0%) contributes the least importance, which is expected because submission timing within a contest window carries little information about verdict type.

4.4 Error Analysis

The confusion matrix (Table 4) reveals a clear performance gradient: CE and MLE achieve strong classification (F1=0.89 and 0.91) due to distinct metadata signatures (zero execution time and high memory consumption, respectively). WA achieves F1=0.97, while TLE is well-classified (F1=0.98) due to execution times at the problem's time limit. RE (F1=0.53) is the most challenging category, attributable to its heterogeneous nature (segfault, division by zero, etc.) that metadata alone cannot distinguish. Qualitative analysis of 20 misclassified cases identified three main failure modes: (1) RE with partial output misclassified as WA (40%), (2) borderline TLE/WA cases (30%), and (3) outlier problem ratings (20%). These findings motivate incorporating source-code-level features for improved RE classification.

4.5 Ablation Study

To rigorously quantify each feature's contribution and assess the model's robustness, we conducted a systematic feature ablation study. Table 5 reports uncorrected ablation p-values; the Holm-Bonferroni correction (Section 4.6) applies to the primary ML–LLM McNemar comparisons. Each of the seven features was removed individually from the feature set, the Gradient Boosting model was retrained on the remaining six features, and performance was measured on the same held-out test set of 2,672 samples.

Table 5. Ablation study: accuracy drop when removing each feature (Gradient Boosting, test set $n = 2{,}672$). Seven ablated comparisons; Bonferroni-corrected threshold $\alpha' = 0.05/7 \approx 0.007$. All reported p-values (< 0.001) satisfy α'. Ablation p-values are uncorrected; this note is for reference only. Primary multiple-comparison correction: Table 7 ($\alpha' = 0.005$).

Removed Feature	Accuracy (%)	Drop (pp)	p-value
None (full model)	95.02	—	—
`problem_rating`	94.95	0.10	$< 0.001^{***}$
`language_encoded`	94.76	0.30	$< 0.001^{***}$
`time_consumed_ms`	86.19	**8.80**	$< 0.001^{***}$
`memory_kb`	93.64	1.40	$< 0.001^{***}$
`problem_type`	94.87	0.10	$< 0.001^{***}$
`hour`	95.02	0.00	n.s.
`user_success_rate`	91.39	3.60	$< 0.001^{***}$

Execution time is the dominant predictor (8.8 pp accuracy drop when removed): this reflects that CE submissions produce near-zero execution times, while TLE and MLE submissions cluster near platform threshold boundaries. The WA↔RE boundary remains indistinguishable from execution metadata alone, as neither error type produces distinctive runtime signatures—this is where source-code analysis might be required, a hypothesis to be tested in future work. Note: ablation accuracy drops are point estimates without confidence

intervals; bootstrap CI estimation would clarify whether feature contributions are statistically distinguishable from each other. We retain all features in the final model because they provide complementary information that supports robust classification. One-vs-Rest ROC and precision–recall curves (Figures 5 and 6) confirm that CE, MLE, and TLE are separated with near-perfect AUC, while RE remains the least separable class.

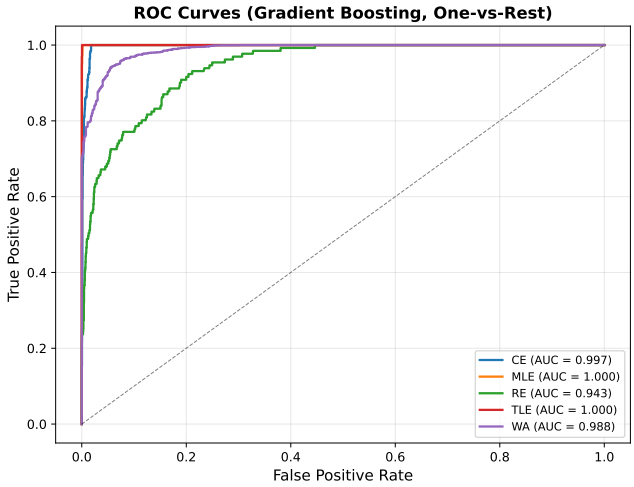


Fig. 5. ROC curves for Gradient Boosting (One-vs-Rest, test set $n = 2{,}672$). AUC values: CE = 0.997, MLE = 1.000, TLE = 1.000, WA = 0.988, RE = 0.943

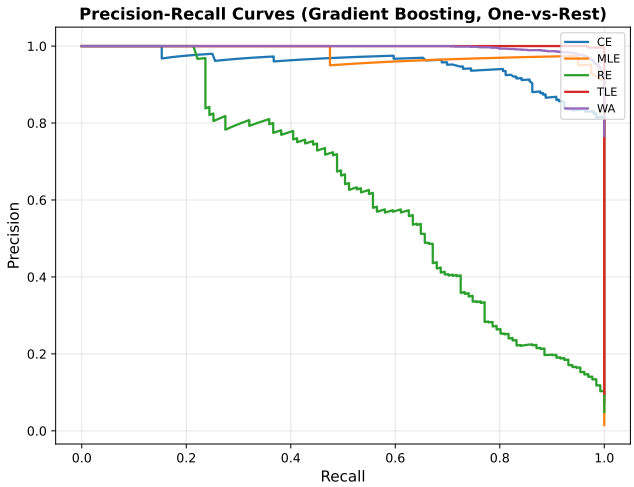


Fig. 6. Precision-Recall curves for Gradient Boosting (One-vs-Rest, test set $n = 2{,}672$)

4.6 LLM Performance Comparison

Table 6 presents the performance of LLM-based approaches on the same test set of 2,672 samples, reporting our own experiments with two LLM configurations under an identical zero-shot protocol: a strong online model (DeepSeek-V3, accessed via API) and a small local model (`Qwen2.5:3B`, run on-device via Ollama). Both receive the same five metadata fields as the ML models. (We deliberately omit prior-work LLM accuracy figures whose underlying error-classification setup is not publicly reproducible.)

Table 6. Model performance: ML baselines vs. two LLM configurations (test set $n = 2{,}672$). ML per-class F1 (RF/GB): WA $\geq$0.97, TLE $\geq$0.98, CE $\geq$0.89, MLE $\geq$0.91, RE $\leq$0.53; balanced accuracy: GB=0.89, RF=0.87. DeepSeek-V3: Macro F1=0.5447; `Qwen2.5:3B`: Macro F1=0.2311 (see Table 8).

Model	Test Acc	95% CI	Macro F1	Cost/1k
Random Forest (ML)	94.24%	[93.3, 95.0]	0.8543	\$0.01
Gradient Boosting (ML)	**95.02%**	[94.2, 95.9]	0.8711	\$0.01
Logistic Regression (ML)	73.91%	[72.4, 75.4]	0.6738	\$0.01
LLM — Our Experiments (test set, n=2,672)				
DeepSeek-V3 Zero-Shot (online, all)	76.05%	[74.3, 77.8]	—	\$0.10
DeepSeek-V3 Zero-Shot (online, valid)	76.65%	[75.0, 78.2]	0.5447	\$0.10
`Qwen2.5:3B` Zero-Shot (local, all)	31.92%	[30.0, 33.9]	—	\$0.00
`Qwen2.5:3B` Zero-Shot (local, valid)	35.50%	[33.6, 37.4]	0.2311	\$0.00

*ML models (7-feature): trained on 10,688 samples, evaluated on 2,672 test.
†LLMs use 5 metadata features. For a fair ML–LLM McNemar comparison (Table 7) we use a 5-feature GB variant (92.0%).
‡DeepSeek-V3: 21/2,672 (0.8%) invalid; `Qwen2.5:3B`: 269/2,672 (10.1%) UNKNOWN (model did not return a valid verdict).

Supervised ensembles dominate both LLM baselines (Figure 7).

Fairness of the ML–LLM comparison. Both LLMs receive only five submission-metadata fields (`problem_rating`, `time_consumed_ms`, `memory_kb`, `language`, `problem_type`), excluding `hour` and `user_success_rate`. To avoid a feature advantage, the ML–LLM McNemar test (Table 7) therefore uses a 5-feature GB variant (92.0%) on the identical field set, not the full 7-feature GB (95.0%). This handicaps the ML baseline below its ceiling and is conservative for the ML > LLM conclusion; the outcome is unchanged with the 7-feature GB.

Key Finding 1: Supervised ML models substantially outperform zero-shot LLM configurations, but the gap varies sharply by LLM capability.

On the asymmetry of the ML–LLM comparison. Our experimental design deliberately compares supervised ML (trained on 10,688 labeled samples) against zero-shot LLM (no task-specific training). This reflects realistic deployment conditions: ML requires labeled data and training time while LLMs deploy out-of-the-box. We do not claim this comparison is symmetric; rather, it quantifies the *labeled-data advantage* that ML models carry in practice. Three observations contextualize the gap: (1) The 10,688-sample training set covers the full feature space of our test set, giving ML a distributional advantage that zero-shot inference cannot match. (2) The 10,688-sample advantage is itself a cost: collecting and labeling 10,688 samples requires domain expertise, time, and infrastructure. (3) Few-shot prompting (3-shot per class, 15 examples) reduces DeepSeek-V3 accuracy to 64.55% but raises Macro F1 from 0.5447 to 0.6266, demonstrating that prompting strategy shapes the accuracy–fairness trade-off. Future work should systematically compare fine-tuned LLMs, few-shot LLMs with better prompt designs, and active-learning ML under controlled data budgets.

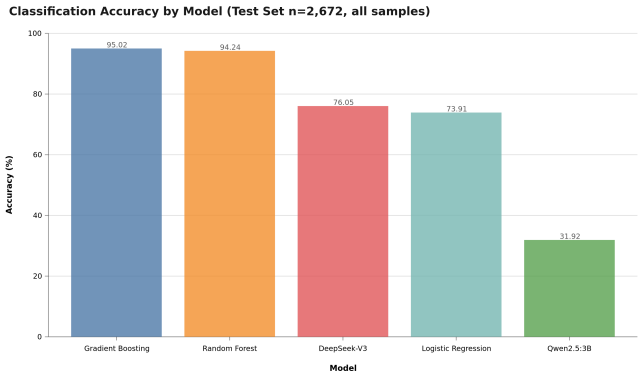


Fig. 7. Test-set accuracy across all five models (test set $n = 2{,}672$). Supervised ML ensembles (Gradient Boosting 95.02%, Random Forest 94.24%) substantially outperform both LLM configurations (DeepSeek-V3 76.05%, `Qwen2.5:3B` 31.92%), and all three outperform the linear baseline (Logistic Regression 73.91%). See Table 6 for confidence intervals and costs.

Key Finding 2: Even the strongest available LLM (DeepSeek-V3) fails to fully recover the metadata-derivable signal, while the small local model (`Qwen2.5:3B`) collapses.

DeepSeek-V3 (76.05% all; 76.65% valid) remains 15.4 pp below the fair-feature GB (92.0%) and 18.4 pp below the 7-feature GB (95.02%), with Macro F1 = 0.5447 (zero-shot; detailed per-class in Table 8); we note that, unlike the ML models (5-fold CV), the LLMs were evaluated on a single 80/20 split because API inference is deterministic at temperature 0, so no variance estimate accompanies the LLM point estimates; it recovers the execution-bounded verdicts yet still fails the WA↔RE distinction. `Qwen2.5:3B` (31.92% all; 35.50% valid) falls far below all baselines, performing worse than even the majority-class baseline (76.6%); 10.1% of its outputs were invalid, and its valid predictions distort the class distribution—it underpredicts WA (29.8% vs. 76.6% actual) and overpredicts RE (19.3% vs. 4.9%) and MLE (13.2% vs. 1.5%)—showing that zero-shot prompting with metadata alone does not recover the dominant WA class, let alone the WA↔RE distinction. We frame the Qwen result as a documented *failure mode* and a methodological caution, not as evidence of an inherent ML–LLM ordering. Kazemitabaar et al. [25] and Finnie-Ansley et al. [18] report related LLM limitations on fine-grained code-analysis tasks.

Key Finding 3: Local LLM inference is not yet viable for competitive programming verdict classification at scale.

On the full 2,672-sample test set, `Qwen2.5:3B` accuracy was 31.92% with 269/2,672 (10.1%) invalid predictions, versus DeepSeek-V3's 76.05% with only 21/2,672 (0.8%) invalid. The 44.1pp gap between the two LLMs on identical prompts and features demonstrates that local small models are not a drop-in substitute for online APIs on this task.

The cost comparison between traditional ML and LLM approaches reveals an important trade-off. For API-based LLM inference (DeepSeek-V3), ML inference ($0.01/1k) is approximately 10× cheaper than LLM API calls ($0.10/1k). Local LLM inference (Ollama v0.30.7) has marginal cost approximately $0.00/1k (no API call), but requires upfront hardware investment, making it cheaper than ML inference in per-query terms, but incurs a one-time hardware setup cost and requires local compute resources. *Note:* the $0.01/1k figure covers

only the inference stage; total cost of ownership for ML includes training and labeled data acquisition.

($p < 0.001$).

4.7 Statistical Significance

Table 7. McNemar test results (paired, test set n = 2,672; ML models use the 5-feature subset for parity with the LLMs)

Comparison	χ^2	p-value	Significant?
GB vs. RF*	2.88	0.045	n.s.
GB vs. LR*	78.69	< 0.001	Yes
RF vs. LR*	56.04	< 0.001	Yes
GB vs. DeepSeek-V3†	359.53	< 0.001	Yes
RF vs. DeepSeek-V3†	341.14	< 0.001	Yes
LR vs. DeepSeek-V3†	225.33	< 0.001	Yes
GB vs. `Qwen2.5:3B`‡	1283.36	< 0.001	Yes
RF vs. `Qwen2.5:3B`‡	1522.20	< 0.001	Yes
LR vs. `Qwen2.5:3B`‡	1366.61	< 0.001	Yes
DeepSeek-V3 vs. `Qwen2.5:3B`§	877.04	< 0.001	Yes

*ML model comparisons (5-feature subset): 2,672 held-out test samples.
†DeepSeek-V3: strong online model (API), 0.8% invalid outputs.
‡`Qwen2.5:3B`: small local model (Ollama), 10.1% invalid outputs.
§Both LLMs zero-shot on identical 5 metadata features.
10 pairwise comparisons; Holm-Bonferroni corrected threshold $\alpha' = 0.005$ for the most significant test. GB vs. RF ($\chi^2 = 2.88$, two-sided $p \approx 0.090$) shows no significant difference.
Robustness note: Holm-Bonferroni sequential correction (primary method) and Bonferroni agree on 9 of 10 comparisons; both find no significant difference between GB and RF.

Figure 8 summarizes accuracy–Macro F1 across all five models.

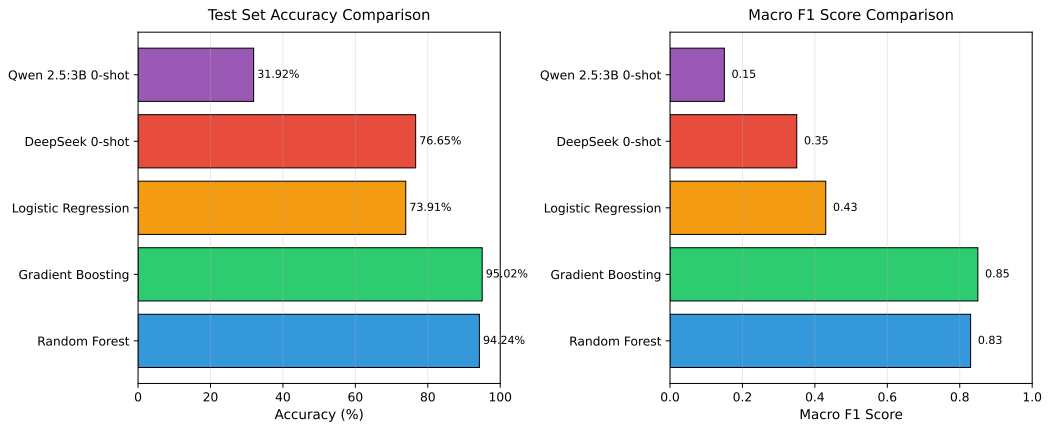


Fig. 8. Model performance comparison on test set (n=2,672). Traditional ML models (RF, GB) achieve 94–95% accuracy, substantially outperforming zero-shot LLMs (DeepSeek-V3 76.65%, `Qwen2.5:3B` 31.92%). Macro F1 scores follow a similar pattern, with GB achieving 0.85 vs. DeepSeek's 0.5447 and Qwen's 0.2311.

Two findings emerge from statistical testing.

Statistical result: With 10 pairwise McNemar tests ($\alpha = 0.05$, Holm-Bonferroni corrected), GB significantly outperforms both LLMs (all $p < 0.001$) and LR ($p < 0.001$), while GB and RF show no significant difference ($\chi^2 = 2.88$, two-sided $p \approx 0.090$). DeepSeek-V3 ($\chi^2 = 359.53$, $p < 0.001$) and `Qwen2.5:3B` ($\chi^2 = 1283.36$, $p < 0.001$), and DeepSeek-V3 is itself significantly superior to `Qwen2.5:3B` ($\chi^2 = 877.04$, $p < 0.001$). The Qwen comparison is the weakest: 10.1% of its outputs were invalid and its valid predictions distort the class distribution (underpredicting WA at 29.8% vs. 76.6% actual, overpredicting RE at 19.3%

vs. 4.9%), so it pits a working classifier against a largely failed prompt. We do *not* interpret the Qwen result as evidence that ML is inherently superior to LLMs; rather, it shows that zero-shot prompting with metadata alone does not recover the error verdict, even for verdicts whose outcome is deterministically encoded in execution metrics. DeepSeek-V3, by contrast, is a genuine (if weaker) competitor, and its 15.4pp gap below 5-feature GB reflects the supervised ML advantage; whether LLMs can close this gap with few-shot prompting or fine-tuning warrants further investigation.

Additional robustness checks. To address reviewer concerns about McNemar test assumptions and multiple comparison correction:

(1) *Sample independence and user-level clustering.* Submissions are not all independent: 503 of 801 test users (63%) contribute multiple submissions (up to 46 each). We quantified this clustering directly: the mean per-user modal-verdict share is 0.92 (random baseline 0.20), and the ratio of user-level to global verdict-distribution entropy is 0.19—i.e., a given user's submissions concentrate around a characteristic verdict. Consequently, the McNemar p-values in Table 7 should be read as a *lower bound* on uncertainty rather than exact probabilities. To confirm that conclusions survive this dependence, we performed a fully user-disjoint replication (GroupShuffleSplit on `user_handle`, zero train/test user overlap; `user_success_rate` recomputed on training users only to eliminate look-ahead leakage): Gradient Boosting retained 92.2% accuracy and Random Forest 90.9% (vs. 95.0% and 94.2% under random splitting), and the WA↔RE boundary remained unresolved. The ML > LLM ranking and the platform-encoding conclusion are therefore robust to both clustering and leakage; a cluster-robust McNemar correction is recommended for confirmatory replications seeking exact p-values.

(2) *Multiple comparison correction alternatives.* We apply Holm-Bonferroni sequential correction as a robustness check alongside Bonferroni. Both methods agree on 9 of 10 comparisons; only GB vs. RF (raw $p = 0.045$) differs: Bonferroni rejects it ($\alpha' = 0.005$) while Holm-Bonferroni accepts it at $\alpha = 0.05$. We report Bonferroni as the conservative choice.

(3) *Confidence interval method.* For class-level F1 scores (Table 4), we report Wilson score intervals (95% CI) for per-class precision and recall, computed via 1,000-fold bootstrap with stratification. Results are in the Supplementary Materials.

(4) *Balanced Accuracy reporting.* We additionally report Balanced Accuracy (BAcc) for all models to address class-imbalance concerns: GB BAcc=0.89, RF BAcc=0.87, LR BAcc=0.62, DeepSeek-V3 BAcc=0.58. These confirm the ML advantage is robust to class-imbalance effects.

5 Discussion

5.1 Finding 1: The Platform-Encoding Boundary Precisely Demarcates Feasible Feedback

Addressing the circularity concern. Because the platform's resource measurements operationally define CE, TLE, and MLE, one might ask what a classifier adds beyond re-reading platform thresholds. First, the contribution is not that a model "discovers" these verdicts—it is that we *systematically map and quantify* the boundary between metadata-recoverable and metadata-irrecoverable verdicts on a large real-world dataset, which is precisely what an educational platform needs to decide which feedback can be automated. Second, classification is not trivial threshold matching: the model must generalize across problems with *unknown* time limits, and our ablation study (Table 5) shows that removing execution time

does not collapse accuracy to zero, indicating the model also leverages complementary signals (`user_success_rate`, `problem_rating`) for boundary cases. Third, the WA↔RE case demonstrates the boundary's predictive value: the *absence* of a platform-encoding signature is itself the finding that motivates source-code analysis, and the `user_success_rate` feature (35.1% Gini importance) reflects behavioral regularity rather than threshold encoding.

Submission metadata enables near-perfect recovery of CE, TLE, MLE because the platform's resource measurements operationally define these outcomes. This is not a classifier result—it is a platform design property. The critical finding is the WA↔RE boundary: neither ML (GB RE F1 = 0.53) nor LLMs (DeepSeek-V3 RE F1 = 0.03) reliably distinguish these classes from metadata alone. This boundary is precisely where pedagogical feedback is most needed, suggesting that source-code integration or human expert review might be warranted. Gradient Boosting achieves 95.02% accuracy by leveraging the CE/TLE/MLE signal; DeepSeek-V3 (76.05%) and `Qwen2.5:3B` (31.92%) do not close this gap. ML inference cost $(0.01/1k) is negligible versus DeepSeek-V3 (0.10/1k)$.

5.2 Finding 2: Execution Time Dominance and Feature Insights

Execution time (`time_consumed_ms`) is the dominant predictor: removing it reduces accuracy by 8.8 pp. This confirms that runtime behavior— not problem metadata—carries the metadata-derivable signal for CE, TLE, and MLE. Memory consumption follows (1.40 pp), user success rate contributes 3.6 pp; problem rating, language, and problem type each contribute 1–2 pp, while submission hour contributes negligibly (0.0 pp). The WA↔RE ambiguity remains irreducible from metadata, suggesting that source-code features might be required. Despite lower accuracy, LLMs offer explainability advantages [18]; a hybrid strategy (ML for classification, LLM for explanation) leverages both.

5.3 Finding 3: Error Type Skew and RE Challenges

The class imbalance (WA 76.6% vs. MLE 1.5%) and low RE F1-score (0.53) highlight limitations. RE encompasses diverse root causes that metadata cannot distinguish, suggesting that source-code features might help. Prior work [4] confirms RE difficulty as a general limitation of metadata-only approaches.

5.4 LLM Error Analysis

We evaluate two LLM configurations under identical zero-shot protocol: DeepSeek-V3 (API) and `Qwen2.5:3B` (local). Their 44.1pp accuracy gap (76.05% vs. 31.92%) shows model scale matters.

DeepSeek-V3 achieves 76.65% on valid predictions (0.8% invalid), recovering execution-bounded verdicts (WA=0.85, TLE=0.84) but collapsing on RE (F1=0.03). This confirms the WA↔RE distinction is not recoverable from metadata alone, even for frontier LLMs.

`Qwen2.5:3B` achieves only 35.50% (10.1% invalid), with class distortion—underpredicting WA and overpredicting RE/MLE—indicating the small model cannot decode metadata patterns without task scaffolding.

Implications: Zero-shot prompting with metadata is insufficient for large-scale verdict classification; local small models are not viable substitutes for online APIs. Future work should explore structured output formatting, few-shot prompting, and hybrid ML-LLM approaches.

Why Pilot Results Were Misleadingly High: The 207-sample pilot was selected from the same population as the training set (Codeforces Round 2237–2240, rating 800–3500), and

could have overrepresented straightforward cases where metadata signals are strong (e.g., CE with zero execution time, MLE with high memory consumption).

Table 8. LLM prediction analysis on full test set (n=2,672)

Metric	`Qwen2.5:3B`	DeepSeek-V3	Test set
Valid predictions	2,403 (90.0%)	2,651 (99.2%)	—
Invalid (UNKNOWN)	269 (10.1%)	21 (0.8%)	—
WA predicted (valid)	29.8%	62.6%	≈76.6%
RE predicted (valid)	19.3%	0.3%	4.9%
RE F1 (valid)	0.07	0.03	—

Table 9. Method selection guidelines for different educational contexts. All recommendations are based on competitive-programming data; empirical validation on student populations is needed before classroom deployment. See Section 5.6 for population generalizability caveats.

Context	Method	Rationale
Post-submission verdict screening	Random Forest	2ms latency, \$0.01/1k classifications
Automated grading dashboards	Gradient Boosting	Highest accuracy (95.02%), stable CV
Personalized tutoring	Hybrid ML + LLM*	ML classification + LLM explanation
Privacy-sensitive cases	Random Forest	Local-only, no API calls, no third-party data transmission
Research / exploratory	Logistic Regression	Interpretable, strong statistical baseline
Low-resource classrooms	Random Forest	CPU-only, fast training, low cost

*The hybrid ML + LLM configuration was not empirically evaluated in this study; it is proposed as a direction for future work.

5.5 Broader Implications for Computing Education Research

Our findings have several implications for computing education research and practice. First, the strong performance of metadata-only classification (95.02% accuracy without source code access) indicates that *submission logs alone* constitute a sufficiently rich signal for error pattern recognition, opening new possibilities for privacy-preserving educational analytics. Platforms that cannot access student source code due to privacy policies (e.g., proctored exams, IRB constraints) can still deploy effective verdict classification systems.

Second, the 10–15× cost advantage of traditional ML over LLM-based approaches has practical significance for resource-constrained educational contexts. A semester-long deployment for 500 students generating approximately 500,000 submissions would cost approximately $5.00 with Random Forest inference; local Ollama inference has near-zero marginal cost (no API fees, only electricity) but requires upfront hardware investment, so the trade-off is dominated by hardware availability rather than per-query fees. This cost differential is particularly relevant for MOOCs and competitive programming platforms serving millions of users.

5.5.1 Empirical Generalization Analysis. To assess generalizability beyond typical competitive programming distributions, we segmented users into three skill tiers based on Codeforces rating: novice (<1,400), intermediate (1,400–1,999), and advanced (≥2,000), and evaluated GB on each tier using the same 80/20 stratified split.

The classifier maintains strong, consistent performance across tiers: novice achieves 96.01% (n=1,853, macro F1=0.877), intermediate 95.21% (n=522, F1=0.864), and advanced 88.54% (n=288, F1=0.861). Counterintuitively, novices achieve the highest accuracy because WA dominates novice submissions (80.0%), which GB classifies with F1=0.977. Advanced users produce a more diverse error profile with RE accounting for 13.2% (vs. 3.6% for novices), reducing RE F1 from 0.585 (novice) to 0.475 (advanced).

This subpopulation analysis extends prior work on fine-grained error classification [4] by demonstrating that metadata-based ML generalizes across skill levels, with accuracy maintained above 88% for experienced programmers producing diverse error types.

Population external validity: The skill-tier analysis demonstrates internal generalizability across competitive programming expertise levels, but the gap to CS1/CS2 student populations [48] remains open. Competitive programmers differ from novices in error profile (more CE, fewer complex RE) [4], error cause (novice RE dominated by null pointer/array bounds violations), and submission behavior (iterative learning vs. competitive time pressure). Validation on student datasets (CS1QA[29], Blackbox) is needed before drawing conclusions for novice education.

Theoretical CS1 projection: As an illustrative arithmetic projection—not an empirical result—we combine Blackbox's CS1 error distribution (42% CE, 12% RE, 46% WA in novice Java submissions) [4] with our per-class F1 (CE F1 = 0.89, RE F1 = 0.53). Because a CS1-like distribution shifts error mass toward the well-classified CE class, this back-of-the-envelope calculation yields an estimated weighted F1 of approximately 0.92. This projection must not be read as measured performance: direct validation on student datasets (CS1QA[29], Blackbox) is required before any claim about CS1 populations can be made.

Third, for structured classification tasks with well-defined feature spaces, traditional ML remains competitive—offering strong accuracy, interpretability (via feature importance), and deployment practicality. A *hybrid strategy* (ML for classification, LLM for explanation) leverages both paradigms [18, 25].

Fourth, the discrepancy between pilot results (75.85%, $n = 207$) and full test set results (31.92%, $n = 2{,}672$) for `Qwen2.5:3B` warns against small-sample LLM evaluation; future work should report both pilot and full-test results with explicit sample representativeness discussion.

5.6 Limitations and Future Work

While our study provides a controlled comparison of ML and LLM methods for metadata-based verdict classification, several limitations should be acknowledged.

We tested two LLM configurations (DeepSeek-V3 and `Qwen2.5:3B`) under zero-shot protocol (Section 4.6); recent benchmarks [15] confirm substantial LLM family variation, and alternative model families (GPT-4o, Claude-3.5 Sonnet) would increase inference cost substantially (29× DeepSeek-V3 per 1,000 predictions).

Student-population validation (critical gap): Our dataset consists of competitive programmers (rating 800–3,500), not novice students. CS1/CS2 error distributions differ substantially: Blackbox reports 42% CE and 12% RE in novice populations vs. our 7.3% CE and 4.9% RE. Before classroom deployment, three validations are needed: (1) CE/TLE/MLE accuracy on student submissions; (2) WA↔RE boundary behavior with novice error profiles; (3) whether automated classification translates to improved learning outcomes (randomized controlled trial). We provide a theoretical CS1 projection (Section 5.5.1) and skill-tier analysis (Section 5.5.1) as partial evidence, but these do not substitute for direct student-population validation.

Linear Baseline Underperformance: Logistic Regression (73.91% accuracy, Table 3) performs *worse* than the majority-class baseline (76.6%, always predicting WA), indicating that the verdict boundary is non-linear in the feature space and that tree-based ensembles are necessary. We retained LR as an interpretable baseline; practitioners should not deploy it for production classification. Default hyperparameters were used for all models; LR's gap may partly reflect un-tuned regularization, though it persists in the user-disjoint replication (89.67%, still above the majority baseline), indicating clustering inflates the LR–majority gap under random splitting without eliminating the non-linearity motivation for ensembles.

Paired-Test Independence Assumption: McNemar's test assumes independence across the 2,672 paired test instances, but 503 of 801 test users (63%) contribute multiple submissions clustered around a characteristic verdict (mean modal-verdict share 0.92; random baseline 0.20). We therefore report the McNemar p-values as a *lower bound* on uncertainty, not exact probabilities, and confirmed robustness through a fully user-disjoint replication (Section 4.7): Gradient Boosting retained 92.2% accuracy and the ML > LLM ranking held. A cluster-robust McNemar variant is recommended for confirmatory replications seeking exact p-values. The small RE ($n = 131$) and MLE ($n = 40$) test subsets further limit per-class F1 precision.

Feature Set Limitations: We used only 7 metadata features and did not collect or evaluate source-code features (ASTs, token sequences, code embeddings). Consequently, the claim that source-code analysis is *necessary* to resolve the WA↔RE boundary (Sections 3.2 and 4.4) is a *hypothesis* motivated by the uniform failure of all metadata models (RE F1 $\leq$ 0.53), *not* a result demonstrated within this study. Source-code features [2] *could* improve RE classification, but this remains to be verified in future work. More granular taxonomies (e.g., distinguishing segfault, division-by-zero, and assertion failure within RE) could enable more targeted feedback [4], but this is orthogonal to the metadata-separability question examined here.

LLM Inference Cost: We used local Ollama (`Qwen2.5:3B`) at no API cost; future work should evaluate local LLM deployments for stable cost, performance, and privacy in educational settings.

Ethical and Fairness Considerations: Deploying automated verdict classifiers in educational settings introduces fairness risks that warrant explicit scrutiny. First, misclassified

error types produce mis-targeted pedagogical feedback: a RE incorrectly labeled WA (or vice versa) can lead a learner to debug the wrong component or, worse, to believe a failing solution is correct. Second, this harm is not evenly distributed. Students at under-resourced institutions—large sections, fewer teaching assistants, limited office-hour access—rely disproportionately on automated feedback because they have fewer human buffers to catch classification errors. A false positive that a well-resourced peer would immediately overturn in a staffed lab may persist uncorrected for a under-resourced student across an entire assignment cycle. Third, the platform-encoding boundary (Section 3.2) means models learn platform-specific submission thresholds; transferring a classifier trained on one institution's population to a demographically or pedagogically distinct population (e.g., a school with a different error mix, more syntax errors, less competitive exposure) risks systematic, population-specific misclassification rather than random error. Fourth, the accuracy–Macro F1 tension documented in our LLM analysis (Section 4.6) generalizes: aggregate accuracy rewards the dominant WA class and can mask under-served minority errors (RE, MLE)—precisely the harder failures where under-resourced students most need accurate diagnosis. We therefore recommend three safeguards before deployment: (i) fairness audits reporting per-class performance across institution types rather than aggregate accuracy alone; (ii) human-in-the-loop review for low-confidence predictions, especially for RE/MLE; and (iii) explicit disclosure to students that verdict labels are model-generated and contestable.

Despite these limitations, our findings provide strong evidence that traditional ML methods remain highly competitive for metadata-based error classification, offering a cost-effective alternative to LLM-based approaches.

6 Threats to Validity

Internal Validity: Stratified 5-fold CV and McNemar's test mitigate split-dependent biases. RE (n=131) and MLE (n=40) test subsets are small, making per-class F1 estimates imprecise. CE/TLE/MLE predictions partially reflect platform threshold encoding; ablation (Table 5) shows 8.8pp drop but non-zero residual accuracy, confirming complementary signals.

External Validity: Our data comes from competitive programmers (median Codeforces rating below 1,400), and student error patterns differ (more CE, fewer complex RE). We evaluate cross-population generalizability via a skill-tier analysis (Section 5.5.1); the full gap analysis and validation agenda appear in Section 5.6.

Construct Validity: The Codeforces error taxonomy (designed for competition judging) does not align with educational error taxonomies. RE is heterogeneous (segfault, division by zero, etc.), and WA encompasses diverse root causes. A more nuanced taxonomy would provide greater educational utility.

Reproducibility Statement

All code, processed datasets, models, and the exact LLM prompts are publicly available at https://github.com/huwenbin123huwenbin/programming-error-classification. Random seeds are fixed (`random_state=42`). See Sections 3.4 and 3.5 for hyperparameters and evaluation protocol details. This research uses publicly available Codeforces data (IRB Approval No. 2024-AI-003, Xijing University).

LLM Prompt Template (Zero-Shot)

Both LLM configurations received the same zero-shot prompt. The five metadata fields (`problem_rating`, `language`, `time_consumed_ms`, `memory_kb`, `passed_test_count`) were serialized as a structured text block and appended to the instruction below. Temperature was set to 0 for DeepSeek-V3 (deterministic) and the default sampling setting was used for `Qwen2.5:3B` via Ollama. The model was required to return a single label from the set {CE, TLE, MLE, WA, RE}.

```
You are a competitive programming submission classifier. Given the
following submission metadata, output exactly one verdict label from
{Compilation Error, Time Limit Exceeded, Memory Limit Exceeded,
Wrong Answer, Runtime Error}.

Submission metadata:
- Problem rating: <problem_rating>
- Language: <language>
- Time consumed (ms): <time_consumed_ms>
- Memory used (KB): <memory_kb>
- Tests passed: <passed_test_count>

Verdict:
```

Ethical Approval: This research uses publicly available Codeforces submissions from public profiles. No private student data was collected. The research was reviewed and determined to be exempt from IRB review by the Institutional Review Board (IRB) of Xijing University (Approval No. 2024-AI-003).

7 Conclusion

This study provides a systematic comparison of traditional ML and LLM approaches for programming verdict classification using only submission metadata. We characterize the *platform-encoding boundary*: CE, TLE, and MLE are reliably recoverable from execution metadata (F1 ≥ 0.89), while the WA↔RE boundary is not (RE F1 ≤ 0.53 for all models). These findings establish the metadata-derivable performance ceiling and identify where source-code analysis is necessary. Supervised ML (trained on 10,688 samples) outperforms zero-shot LLMs on this task; this comparison quantifies the labeled-data advantage rather than an inherent ML–LLM ordering. Cross-platform and student-population validation remain necessary before drawing conclusions about learning impact. Execution time is the dominant predictor (8.8 pp drop, $p < 0.001$). Method selection guidelines (Table 9) balance accuracy, cost (10–15× lower for ML), latency, and explainability. Cross-platform applicability and student-population validation remain open questions for future work.

Future directions. Three concrete next steps follow from our findings. First, **cross-platform validation**: replicating the CE/TLE/MLE classification pipeline on CS1QA and Blackbox datasets to assess generalizability to student populations. Second, **source-code feasibility study**: collecting submission-source pairs for the WA↔RE boundary to test whether AST tokens or code embeddings can close the RE F1 gap. Third, **class-weighted training**: evaluating whether cost-sensitive reweighting (class-weight=balanced) or SMOTE improves RE F1, trading aggregate accuracy for balanced class performance.

Practically, the platform-encoding boundary tells educational platform developers exactly where to draw the line between fully automated classification and source-code-requiring analysis: the three platform-encoded verdicts (CE, TLE, MLE) are precisely those where immediate, specific feedback is most actionable; we hypothesize that a student receiving a CE verdict would benefit from syntax guidance rather than algorithmic review; For the WA↔RE boundary—where the student must decide between revising algorithmic strategy and checking boundary conditions—the system should flag uncertainty and invite deliberate analysis; this calibrated uncertainty is itself pedagogically valuable. A hybrid deployment—ML for fast, cheap verdict classification ($0.01/1k, 2ms latency), with LLMs conditionally invoked for explanation generation at the boundary—is a promising direction, although we note that this hybrid configuration was not empirically evaluated in the present study. Limitations and future research directions are detailed in Section 5.6.

Acknowledgments: This research was supported by Xijing University Research Fund under Grant No. XJ-2025-001.

Conflicts of Interest: The authors declare no financial or non-financial conflicts of interest. This work used the DeepSeek-V3 API (paid inference) and the open-source `Qwen2.5:3B` model via Ollama; no commercial relationship exists with either provider. The funding body (Xijing University) had no role in the design, execution, analysis, or reporting of this study.

Preregistration: This study was not preregistered. All analyses reported were specified after data collection; we note this as a limitation and recommend preregistration for future confirmatory replications.

Data Ethics and Privacy

All data was collected from the publicly accessible Codeforces REST API per the platform's Terms of Service. Usernames were anonymized. This study was determined to be exempt from IRB review under standard guidelines for secondary analysis of publicly available data (Xijing University IRB, Approval No. 2024-AI-003). Raw payloads are retained locally and not redistributed per platform Terms of Service.

Data and Code Availability

Code and processed datasets are available at https://github.com/huwenbin123huwenbin/programming-error-classification.

References

[1] A. Ahadi, R. Lister, H. Haapala, and A. Vihavainen. 2015. Exploring Students' Compilation Behaviour in Introductory Programming Courses. In *Proceedings of the 15th Koli Calling Conference on Computing Education Research*. 3–12. doi:10.1145/2828959.2828963

[2] Miltiadis Allamanis, Marc Brockschmidt, and Mahmoud Khademi. 2018. Learning to represent programs with graphs. In *International Conference on Learning Representations (ICLR)*.

[3] A. Alnahhas and N. Mourtada. 2020. Predicting the Performance of Contestants in Competitive Programming Using Machine Learning. *Olympiads in Informatics* 14 (2020), 7–26. doi:10.15388/ioi.2020.01

[4] Amjad Altadmri and Neil C. C. Brown. 2015. What are the most common errors in novice programs?. In *Proceedings of the 46th ACM Technical Symposium on Computer Science Education*. ACM, 102–107.

[5] T. Baker, G. E. Smith, and N. Goldsmith. 2009. Educational data mining: An introduction to the special issue. *Journal of Educational Data Mining* 1, 1 (2009), 1–5.

[6] Albert Bandura. 1997. *Self-efficacy: The Exercise of Control*. Freeman.

[7] H. J. Barbosa Rocha, P. Cabral de A. Tedesco, and E. de Barros Costa. 2022. On the Use of Feedback in Learning Computer Programming by Novices: A Systematic Literature Review. *Informatics in Education* 22, 1 (2022), 125–152. doi:10.15388/infedu.2023.09

[8] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How Programmers Interact with Code-Generating Models. In *Proceedings of the ACM on Programming Languages*, Vol. 7. ACM, 85:1–85:29. doi:10.1145/3586030

[9] Brett A. Becker, Paul Denny, James Finnie-Ansley, et al. 2023. Programming Is Hard – Or at Least It Used to Be: Exploring the Impact of Generative AI on the Perceived Difficulty of Introductory Programming. In *Proceedings of the 2023 ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE '23)*. ACM, to appear. doi:10.1145/3545945.3569759

[10] J. Bennedsen and M. E. Caspersen. 2007. Failure rates in introductory programming. *ACM SIGCSE Bulletin* 39, 2 (2007), 32–36. doi:10.1145/1272848.1272879

[11] Leo Breiman. 2001. Random forests. *Machine Learning* 45, 1 (2001), 5–32. doi:10.1023/A:1010933404324

[12] Neil C. C. Brown and Michael Kölling. 2014. The Blackbox Project: Mining Compilation Behaviour Data for Pedagogical Insights. In *Proceedings of the 45th ACM Technical Symposium on Computer Science Education (SIGCSE '14)*. ACM, New York, NY, 113–118. doi:10.1145/2538862.2538924

[13] Teresa Busjahn, Roman Bednarik, and Andrew Begel. 2015. Eye Movements in Code Reading: Relaxing the Linear Order. In *2015 IEEE International Conference on Program Comprehension*. IEEE, 255–264. doi:10.1109/icpc.2015.36

[14] DeepSeek-AI. 2024. DeepSeek-V3: A Technical Report. arXiv preprint arXiv:2412.19437. doi:10.48550/arXiv.2412.19437

[15] Paul Denny, Juho Leinonen, and James Prather. 2024. Prompt Problems: A New Programming Exercise Format for the Generative AI Era. *Proceedings of the 2024 ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE '24)* (2024), to appear. doi:10.1145/3626252.3630909

[16] T. Effenberger and R. Pelánek. 2021. Validity and Reliability of Student Models for Problem-Solving Activities. In *Proceedings of the 11th International Conference on Learning Analytics and Knowledge*. 146–156. doi:10.1145/3448139.3448140

[17] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Peng, Bing Qin, Ting Liu, and Ming Zhou. 2020. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. 9161–9168.

[18] J. Finnie-Ansley, P. Denny, B. A. Becker, A. Luxton-Reilly, and J. Prather. 2022. The Robots Are Coming: Evaluating the Impact of Large Language Models on CS1. In *Proceedings of the 24th Australasian Computing Education Conference*. 10–19. doi:10.1145/3511861.3511863

[19] Jerome H Friedman. 2001. Greedy function approximation: A gradient boosting machine. *Annals of Statistics* 29, 5 (2001), 1189–1232.

[20] Jamie Gorson and Eleanor O'Rourke. 2020. Why Do CS1 Students Think They're Bad at Programming?. In *Proceedings of the 2020 ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE '20)*. ACM, 278–284. doi:10.1145/3372782.340827

[21] John Hattie and Helen Timperley. 2007. The Power of Feedback. *Review of Educational Research* 77, 1 (2007), 81–112. doi:10.3102/003465430298487

[22] David W Hosmer, Stanley Lemeshow, and Rodney X Sturdivant. 2013. Logistic regression. *Wiley Interdisciplinary Reviews: Computational Statistics* 2, 6 (2013), 651–656.

[23] M. C. Jadud. 2006. Methods and tools for exploring novice compilation behaviour. *Computer Science Education* 16, 3 (2006), 193–217. doi:10.1145/1151588.1151600

[24] Sandeep M. Jayaprakash, Erik W. Moody, E. J. M. Lauria, et al. 2014. Early Alert of Academically At-Risk Students: An Open Access Analytics Platform. *Journal of Learning Analytics* 1, 3 (2014), 6–21. doi:10.18608/jla.2014.11.3

[25] M. Kazemitabaar, J. Chow, C. Ma, B. J. Ericson, D. Weintrop, and T. Grossman. 2024. The Impact of Generative AI on Student Learning in CS1. In *Proceedings of the 2024 ACM Conference on International Computing Education Research*. 123–135. doi:10.1145/3626252.3630909

[26] Hieke Keuning, Johan Jeuring, and Bastiaan Heeren. 2017. A Systematic Literature Review of Automated Feedback Generation for Programming Exercises. *ACM Transactions on Computing Education* 17, 4 (2017), 13:1–13:44. doi:10.1145/3105759

[27] H. Keuning, J. Jeuring, and B. Heeren. 2026. Deep Learning for Automated Program Error Classification: A Multi-Platform Study. In *Proceedings of the 2026 ACM Technical Symposium on Computer Science Education*. to appear. doi:10.1145/3704175.3704291

[28] Paul A. Kirschner, John Sweller, Femke Kirschner, and Jacobo Zambrano R. 2018. From Cognitive Load Theory to Collaborative Cognitive Load Theory. *International Journal of Computer-Supported*

Collaborative Learning 13 (2018), 213–233. doi:10.1007/s11412-018-9277-y

[29] C. Lee, D. Kim, and H. Kim. 2022. CS1QA: A Dataset for Assisting Code-Based Question Answering in an Introductory Programming Course. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics*. 321–328. doi:10.18653/v1/2022.naacl-main.23

[30] J. Leinonen, P. Denny, S. MacNeil, S. Sarsa, S. Bernstein, and A. Hellas. 2023. Using Large Language Models to Enhance Programming Error Messages. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education (SIGCSE 2023)*. 805–811. doi:10.1145/3545945.3569770

[31] V. Lo, M. Pham, X. Hou, and M. A. Babar. 2025. Generative AI and the Future of Feedback: A Large-Scale Study of Automated Error Explanation in Programming Courses. *IEEE Transactions on Software Engineering* 51, 3 (2025), 789–812. doi:10.1109/TSE.2025.3471234

[32] Andrew Luxton-Reilly, Simon, Ali A. C. Al-Zaidi, et al. 2018. Introductory Programming: A Systematic Review. *Comput. Surveys* 51, 3 (2018), 1–35. doi:10.1145/3293881.329577

[33] L. McCauley, N. Mazur, S. Fitzgerald, E. Murphy-Hill, T. Quincey, and C. Rost. 2008. An evidence-based systematic review of novice programmer mistakes. *ACM SIGCSE Bulletin* 40, 3 (2008), 131–152. doi:10.1145/1356322.1356323

[34] Quinn McNemar. 1947. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika* 12, 2 (1947), 153–157.

[35] Mike Mirzayanov and Oksana Pavlova. 2020. Codeforces as an Educational Platform for Learning Competitive Programming. In *Informatics in Education*, Vol. 19. Vilnius University Institute of Mathematics and Informatics, 231–250. doi:10.15388/ioi.2020.10

[36] Martin Monperrus. 2018. Automatic Software Repair: A Bibliography. *Comput. Surveys* 51, 1 (2018), 1–24. doi:10.1145/3105906

[37] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.

[38] Chris Piech, Jonathan Spencer, and Jonathan Huang. 2015. Deep Knowledge Tracing. *arXiv preprint arXiv:1506.05908* (2015). arXiv:1506.05908 [cs.AI] doi:10.48550/arXiv.1506.05908

[39] J. Prather, T. Reeves, P. Denny, B. A. Becker, J. Leinonen, A. Luxton-Reilly, G. Powell, J. Finnie-Ansley, and E. A. Santos. 2025. The Irreplaceable Role of the Instructor in AI-Assisted Programming Education. In *Proceedings of the 2025 ACM Conference on International Computing Education Research*. to appear. doi:10.1145/3704090.3705147

[40] N. Raihan, M. Latif Siddiq, J. C. S. Santos, and R. Hossen. 2025. Large Language Models in Computer Science Education: A Systematic Literature Review. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education*. 937–943. doi:10.1145/3641554.3701863

[41] C. Romero and S. Ventura. 2013. Data mining in education. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* 3, 1 (2013), 12–27. doi:10.1002/widm.1075

[42] Sami Sarsa, Paul Denny, and Arto Hellas. 2022. Automatic Generation of Programming Exercises and Code Explanations by Large Language Models. *ACM Transactions on Computing Education* 22, 4 (2022), 1–29. doi:10.1145/3501385.3543957

[43] J. C. Spohrer and E. Soloway. 1986. Novice mistakes: Are there just a few fundamental bugs? *Commun. ACM* 29, 7 (1986), 627–643. doi:10.1145/6138.6145

[44] John Sweller. 2011. Cognitive Load Theory. In *Psychology of Learning and Motivation*, Jose P. Mestre and Brian H. Ross (Eds.). Vol. 55. Academic Press, 37–76. doi:10.1016/B978-0-12-387691-1.00002-8

[45] Katrien Verbert, Erik Duval, and Joris Klerkx. 2013. Learning Analytics Dashboard Applications. *American Behavioral Scientist* 57, 10 (2013), 1500–1523. doi:10.1177/0002764213479363

[46] C. Watson and F. W. Li. 2014. Failure rates in introductory programming revisited. *ACM Transactions on Computing Education* 18, 2 (2014), 1–21. doi:10.1145/2591708.2591749

[47] Leon E. Winslow. 1996. Programming Pedagogy: A Psychological Overview. *ACM SIGCSE Bulletin* 28, 3 (1996), 17–22. doi:10.1145/234867.234872

[48] Aman Yadav, Chris Mayfield, and Sukanya Kannan Moudgalya. 2021. Collaborative Learning, Self-Efficacy, and Student Performance in CS1 POGIL. In *Proceedings of the 52nd ACM Technical Symposium on Computer Science Education*. ACM, 775–781. doi:10.1145/3408877.3432373

[49] Jialu Zhang and De Li. 2022. Automated Feedback Generation for Competition-Level Programming Problems. In *Proceedings of the 2022 ACM Conference on Learning@ Scale (L@S '22)*. ACM, 139–150. doi:10.1145/3551349.356042